

B-SyncText

Document de conception

École Nationale Supérieure Polytechnique de Yaoundé
Département de Génie Informatique

Table des matières

Introduction **2**

1 Méthodologie **3**

1.1 Démarche . . . 3

1.2 Approche . . . 3

1.3 Identifiants . . . 3

1.4 Outils . . . 3

2 Analyse **4**

2.1 Acteurs . . . 4

2.2 Besoins fonctionnels . . . 4

2.3 Contraintes du sujet et d'hébergement . . . 5

2.4 Besoins non fonctionnels . . . 7

2.5 Hypothèses d'interprétation retenues . . . 7

2.6 Cas d'utilisation . . . 9

2.7 Parcours . . . 10

3 Conception **11**

3.1 Conception architecturale . . . 11

3.2 Conception détaillée . . . 17

4 Choix technologiques **23**

Conclusion **24**

Références **25**

Introduction

Contexte

Une association étudiante publie chaque année un journal numérique, écrit en commun par plusieurs centaines de rédacteurs, souvent sur les mêmes textes et parfois au même endroit. À la parution, le journal est diffusé par un lien de téléchargement annoncé à une heure précise et limité à cinq cents exemplaires. L'équipe continue de corriger après l'annonce : une correction faite après la parution doit se retrouver dans les exemplaires téléchargés ensuite, sans changer de lien [1].

Problème

Le sujet demande de concevoir, construire et déployer sur AWS une application web qui tient ce scénario, puis de prouver qu'elle respecte six contraintes. Trois difficultés en découlent :

— un compteur de téléchargements lu puis réécrit par des milliers de requêtes simultanées laisse tôt ou tard passer un exemplaire de trop ;
— deux rédacteurs qui écrivent au même endroit au même instant produisent deux textes qu'il faut fusionner sans rien perdre ;
— un pic de 15 000 visiteurs en une minute ne doit pas ralentir les 300 rédacteurs qui écrivent pendant ce temps.

La question traitée est : *comment construire, avec des services simples et peu coûteux, une application dont on peut démontrer qu'elle ne délivre jamais un exemplaire de trop, ne perd jamais une modification acceptée et tient le pic de la parution ?*

Objectifs

Par ordre de priorité :

1. tenir les deux contraintes éliminatoires (exactement N téléchargements ; aucune modification acceptée perdue) et le prouver sur l'application déployée ;
2. tenir les quatre autres contraintes (document à jour au moment du clic, crédit journalier, pic de la parution, information rapide de l'épuisement) ;
3. rester simple : peu de composants, chacun justifié [1] ;
4. dépenser le moins possible et pouvoir déployer, tester et supprimer l'ensemble par des commandes documentées.

Périmètre

L'application s'appelle **B-SyncText**. Elle permet de créer et partager des documents, d'écrire à plusieurs en temps réel avec présence et curseurs, de consulter et restaurer des versions, de créer des liens de téléchargement en PDF à nombre d'exemplaires limité, et elle applique un crédit d'écriture quotidien et un blocage des abus. L'association dispose d'un tableau de bord et le créateur d'un lien est prévenu par e-mail quand le lien est épuisé. Il n'y a pas de vrais comptes : un nom et une adresse e-mail suffisent à identifier un rédacteur, et les visiteurs n'ont aucune identité à donner. La mise en forme avancée (images, tableaux), la modération, l'application mobile native et le nom de domaine personnalisé sont hors périmètre.

Plan

La section 1 décrit la méthode. La section 2 analyse les besoins : acteurs, besoins, contraintes, hypothèses, cas d'utilisation. La section 3, *Conception*, comprend deux parties : la conception architecturale (composants, services AWS, mécanismes par contrainte, dimensionnement, sécurité,

coût) et la conception détaillée (classes, données, états, séquences, identité visuelle). La section 4 récapitule les choix technologiques et leurs alternatives.

1 Méthodologie

1.1 Démarche

Le travail suit quatre étapes, parcourues dans l'ordre mais avec des retours :

1. **Analyse** : lire le sujet, en tirer les besoins, les contraintes et les points ouverts, et fixer pour chacun un objectif mesurable (section 2).
2. **Conception** : choisir l'architecture, les services et les technologies, en écrivant pour chaque choix l'alternative écartée et la raison (sections 3 et 4).
3. **Réalisation** : coder l'application et décrire toute l'infrastructure en code, pour la créer et la supprimer par une commande.
4. **Tests sur AWS** : prouver chaque contrainte sur l'application déployée, avec des fichiers de résultats bruts datés.

1.2 Approche

Trois approches sont possibles pour concevoir un logiciel. L'approche *orientée solution* part d'un outil existant. L'approche *orientée méthode* s'appuie sur une démarche formalisée, ici le langage de modélisation UML (*Unified Modeling Language*). L'approche *hybride* combine les deux : on modélise d'abord, puis on associe des outils. C'est l'approche retenue, car le sujet exige que chaque choix soit compris et justifié : le modèle donne les arguments, les outils les réalisent [2, 3].

1.3 Identifiants

Chaque exigence porte un identifiant stable, cité d'un document à l'autre, ce qui permet de suivre une contrainte du sujet jusqu'à son test.

Identifiant	Signification
C-1 à C-6	contraintes numérotées du sujet
BF-xx, BNF-xx	besoin fonctionnel, besoin non fonctionnel (avec objectif mesurable)
CH-xx	contrainte d'hébergement et de projet
H-xx	hypothèse d'interprétation d'un point ouvert du sujet
UC-xx	cas d'utilisation
T-xx	test (rapport de test)

1.4 Outils

Outil	Usage
PlantUML	diagrammes UML (classes, états-transitions, séquences, cas d'utilisation, composants, déploiement)
CloudFormation	description de toute l'infrastructure AWS [4]
Vitest, scripts Node.js, k6	tests unitaires, de concurrence et de charge [5]
LaTeX	rédaction du dossier et du rapport de test
Git	historique du dépôt, un commit par étape

2 Analyse

2.1 Acteurs

Un *acteur* est une personne ou un système extérieur qui interagit avec l'application.

Acteur	Rôle	Identification
Rédacteur	entre avec un nom et un e-mail, écrit dans les documents auxquels il a accès	fiche créée à la première entrée, identifiée par l'e-mail, sans mot de passe ni vérification
Propriétaire	rédacteur qui gère les accès d'un document ; le créateur est le propriétaire *principal*	idem
Visiteur	télécharge le journal par un lien	aucune identité demandée
Association	consulte l'état de l'application (tableau de bord)	code administrateur
Système	tâches sans intervention humaine : recharge du crédit, levée des blocages, détection des abus, e-mail de lien épuisé	—

Le visiteur et l'association n'ont pas de fiche dans les données : le visiteur reste anonyme, l'association se reconnaît à son code.

2.2 Besoins fonctionnels

Priorité : **E** essentiel (exigé par le sujet), **I** important, **S** souhaitable.

ID	Besoin	Prio
BF-01	Entrer avec un nom et un e-mail, sans mot de passe ; l'e-mail identifie le rédacteur	E
BF-02	Créer un document, en choisissant le format de numérotation de ses versions	E
BF-03	Donner accès à un document par e-mail ou par lien d'invitation (BF-23)	E
BF-04	Lister les documents auxquels on a accès	E
BF-05	Écrire à plusieurs en même temps ; voir presque aussitôt les modifications des autres ; savoir si ses propres modifications sont enregistrées	E
BF-06	Voir qui est présent et où chacun écrit	I
BF-07	Consulter les versions précédentes	I
BF-08	Restaurer une version précédente	I
BF-09	Créer un lien de téléchargement en choisissant le nombre de téléchargements autorisés	E
BF-10	Télécharger le document en PDF par le lien, sans identité	E
BF-11	Le PDF est le document au moment du téléchargement, selon le mode du document (dernière version déclarée, ou état au clic)	E
BF-12	Quand le nombre est atteint, tout nouveau clic reçoit « épuisé » sans lire le document	E
BF-13	Pendant un pic, chaque visiteur sait s'il télécharge, doit patienter ou si le lien est épuisé	E
BF-14	Crédit d'écriture de 10 000 caractères par rédacteur et par jour ; crédit épuisé = plus d'écriture jusqu'à la recharge	E
BF-15	Voir son crédit restant	I
BF-16	Repérer et bloquer pour un temps une personne qui s'acharne contre les limites	E
BF-17	Tableau de bord : documents ouverts, liens actifs avec téléchargements restants, personnes bloquées	E
BF-18	Prévenir par e-mail le créateur d'un lien arrivé à son dernier téléchargement	E
BF-19	Voir ses liens et leurs téléchargements restants	S
BF-20	Numéroter chaque exemplaire (« exemplaire 137 / 500 ») et y inscrire la version	I
BF-21	Propriétaires multiples : le créateur est principal ; seul lui retire le rôle à un autre	S
BF-23	Lien d'invitation en écriture, régénérable (l'ancien cesse de fonctionner)	I

2.3 Contraintes du sujet et d'hébergement

Les contraintes C-1 et C-2 sont *éliminatoires* : une application qui ne les respecte pas est jugée non fonctionnelle, même rapide [1].

ID	Contrainte	Ce qu'il faut prouver
C-1	un lien permet *exactement* le nombre de téléchargements autorisé, quel que soit le nombre de clics simultanés	nombre d'exemplaires délivrés $= N$, jamais $N + 1$
C-2	aucune modification acceptée n'est perdue, tous finissent par voir le même document, même au même endroit	textes identiques chez tous les clients, contenant chaque modification acceptée
C-3	un lien créé à T et utilisé à T_1 donne le document tel qu'il est à T_1, complet et cohérent	modifier puis télécharger : le PDF contient la modification
C-4	un rédacteur n'écrit jamais plus que son crédit du jour, même depuis plusieurs appareils	total accepté $\leq 10\,000$ avec plusieurs sessions
C-5	15 000 clics dont 80 % dans la première minute, pendant que 300 rédacteurs écrivent : pas de panne, personne ignoré	100 % des visiteurs reçoivent un état
C-6	après le dernier exemplaire, les visiteurs encore présents l'apprennent vite et le service cesse d'être délivré	délai d'information ; aucun PDF généré après épuisement

Le sujet impose aussi des conditions d'hébergement (CH) :

ID	Contrainte
CH-01, CH-02	déploiement sur AWS, compte personnel en plan gratuit, une seule région : **eu-north-1** (Stockholm)
CH-03	aucune clé d'accès AWS dans le code ou le dépôt
CH-04	chaque composant n'a que les droits dont il a besoin
CH-05	bonnes pratiques du cours appliquées [6]
CH-06	les fichiers à télécharger (PDF générés) sont conservés dans S3
CH-09	livrables : dépôt, dossier de conception, rapport de test, description de l'API, preuve de coût

2.4 Besoins non fonctionnels

ID	Qualité	Objectif mesurable	Lié à
BNF-01	exactitude des liens	**0** exemplaire au-delà de N, sur 100 % des essais, avec au moins 1 000 demandes simultanées ($N = 1$ et $N = 500$)	C-1
BNF-02	convergence	**0** modification acceptée perdue ; textes identiques au plus **5 s** après la fin des écritures ; 50 clients au même endroit	C-2
BNF-03	réactivité de l'édition	modification visible chez les autres en moins de 500 ms (95 %) hors pic, 1 s pendant le pic	BF-05
BNF-04	fraîcheur du PDF	le PDF contient toute modification acceptée avant la demande et correspond à un état réel	C-3
BNF-05	respect du crédit	**0** caractère accepté au-delà de 10 000 par jour, avec 3 sessions simultanées	C-4
BNF-06	tenue du pic	**100 %** des visiteurs reçoivent un état ; p95 < 2 s ; **0** erreur serveur ; les 300 rédacteurs continuent d'écrire	C-5
BNF-07	information d'épuisement	visiteur en attente informé en moins de **5 s** ; plus de lecture ni de PDF après épuisement	C-6
BNF-08	lutte contre l'abus	blocage après 20 refus en une minute ; aucun visiteur légitime bloqué	BF-16
BNF-09	tableau de bord	données à jour à 10 s près	BF-17
BNF-10	notification	**exactement un** e-mail par lien épuisé	BF-18
BNF-11	coût	dépense minimale, coût quasi nul au repos, coût des tests mesuré	—
BNF-12	exploitabilité	déployer, tester et supprimer par commandes documentées	livrable 1
BNF-13	explicabilité	chaque choix justifié, composants peu nombreux	sujet
BNF-14	compatibilité	Chrome, Firefox, Edge et Safari récents ; ordinateur et téléphone ; interface en français	—

2.5 Hypothèses d'interprétation retenues

Le sujet laisse des points ouverts. Voici l'interprétation retenue pour chacun.

ID	Point ouvert	Interprétation retenue
H-01	qui est un rédacteur sans compte ?	« sans compte » veut dire sans mot de passe ni vérification, pas sans enregistrement : une fiche par e-mail (nom, crédit, blocage). L'e-mail n'étant pas vérifié, l'usurpation est une limite assumée du sujet
H-02	que consomme le crédit ?	chaque caractère *inséré* ; effacer ne rend pas de crédit, sinon écrire puis effacer en boucle contournerait la limite ; la restauration ne consomme rien
H-03	heure de recharge	à minuit, heure locale de l'association (UTC+1), réglable au déploiement
H-04	insertion plus longue que le crédit restant	refusée *en entier*, avec le crédit restant et l'heure de recharge : couper le texte ferait payer un morceau que le rédacteur voulait peut-être effacer
H-05	qui peut donner accès ?	les propriétaires ; le créateur est principal et seul peut retirer le rôle à un autre
H-06	qu'est-ce qu'une version ?	un instantané numéroté, créé à la main par un membre (nom facultatif), avant une restauration, ou automatiquement quand un PDF est généré en mode « temps réel » ; restaurer est réservé aux propriétaires ; l'historique n'est jamais réécrit
H-07	quand un exemplaire est-il « téléchargé » ?	il est *réservé* 60 s au visiteur qui reçoit une adresse de fichier à usage unique ; il est *compté* quand le fichier est demandé avec cette adresse ; une réservation non utilisée revient dans le stock. « Patientez » couvre tous les exemplaires réservés par d'autres, ou une saturation
H-08	contenu du PDF	titre, texte, date d'édition, numéro de version et numéro d'exemplaire, sur une page de journal
H-09	qui est l'association ?	un code administrateur généré au déploiement, rangé dans SSM
H-10	qu'est-ce que « s'acharner » ?	un rédacteur qui totalise 20 refus en une minute est bloqué 15 minutes, levée automatique ; les visiteurs ne sont jamais bloqués par adresse IP, car beaucoup en partagent une
H-11	forme du lien	adresse stable à identifiant aléatoire non devinable ; N entre 1 et 10 000 ; états actif puis épuisé (définitif)
H-12	taille et forme d'un document	mise en forme de base (titres, gras, italique, souligné, alignement, listes) rendue aussi dans le PDF ; taille maximale 200 000 caractères, soit environ 80 pages
H-13	« documents ouverts »	documents où au moins un rédacteur est connecté
H-14	répartition des 300 rédacteurs	30 documents de 10 rédacteurs ; essai séparé de 50 rédacteurs au même endroit pour C-2

Le PDF dépend du *mode de téléchargement* du document, que tout rédacteur membre peut changer : en mode **dernière version**, le PDF est la dernière version déclarée par un rédacteur,

pour qu'un lecteur ne reçoive pas un texte à moitié écrit ; en mode **temps réel**, il est le document au moment du clic (c'est ce mode qui démontre C-3).

2.6 Cas d'utilisation

Un *cas d'utilisation* décrit un but qu'un acteur atteint avec l'application.

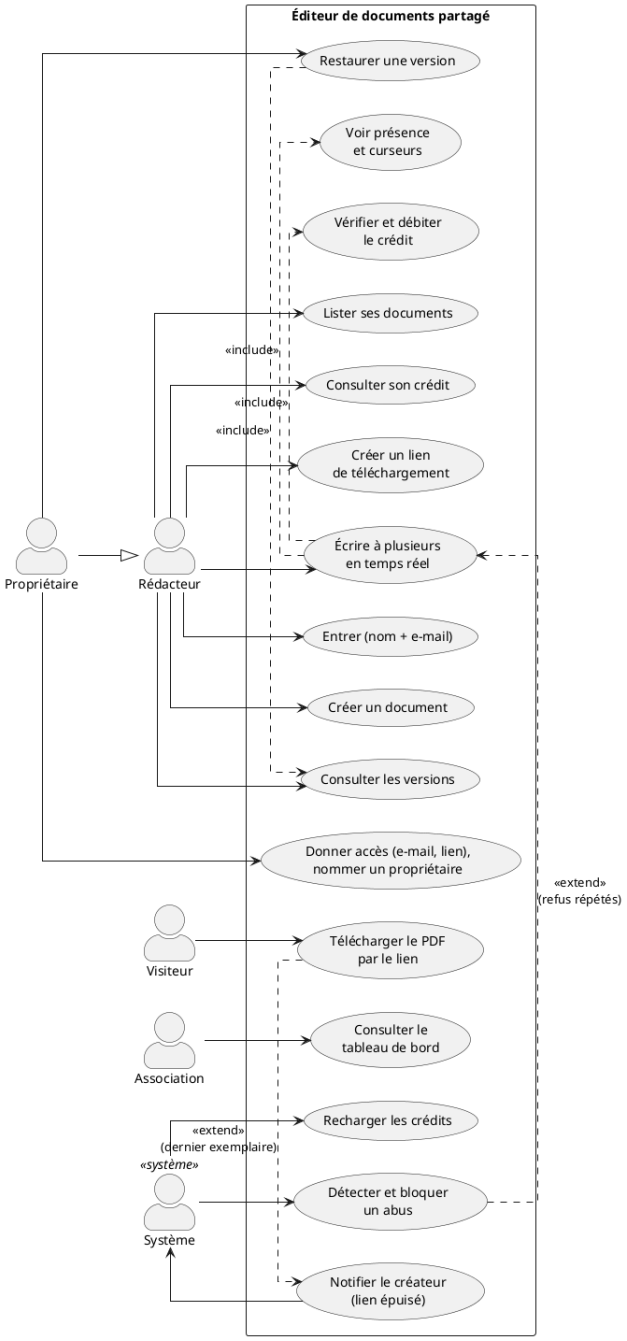


Figure 1 – Cas d'utilisation de B-SyncText

2.6.1 Fiches des cas principaux

UC-05 — Écrire à plusieurs en temps réel. Acteur : rédacteur membre, non bloqué. Une modification est *acceptée* quand le serveur l'a enregistrée, pas avant.

1. Le rédacteur ouvre le document : il reçoit le contenu à jour et la liste des présents.
2. Il écrit ; chaque insertion est vérifiée et débitée sur son crédit.

3. Le serveur accepte et enregistre ; l'indicateur passe de « Synchronisation » à « Enregistré ».
4. La modification apparaît chez les autres presque aussitôt.
5. Si deux rédacteurs écrivent au même endroit au même instant, les deux modifications sont gardées et tous finissent avec le même texte (C-2).

Alternatives : *crédit insuffisant*, l'insertion est refusée en entier avec le crédit restant et l'heure de recharge ; *plusieurs appareils*, le crédit est commun ; *connexion perdue*, un bandeau le signale, les modifications attendent sur l'appareil et sont vérifiées au retour comme les autres ; *refus répétés*, blocage (UC-15).

UC-11 — Télécharger le PDF par le lien. Acteur : visiteur, sans identité.

1. Il ouvre le lien : la page demande aussitôt un exemplaire.
2. Il reçoit un **état** : « prêt » (un exemplaire lui est réservé 60 s, avec une adresse de fichier à usage unique), « patientez » (la page réessaie seule) ou « épuisé ».
3. Le navigateur demande le fichier avec l'adresse à usage unique : l'exemplaire est *compté* et le visiteur reçoit le PDF.

Alternatives : téléchargement automatique bloqué par le navigateur, un bouton « Télécharger » utilise la même adresse ; onglet fermé, la réservation expire en 60 s et l'exemplaire revient dans le stock ; lien inconnu, message « lien introuvable » ; épuisement pendant l'attente, la page passe d'elle-même à « épuisé » en quelques secondes (C-6). Règles : jamais plus de N exemplaires comptés, avec *compté* + *réservé* $\leq N$ à tout instant ; une adresse de fichier ne sert qu'une fois ; une saturation devient « patientez », jamais une erreur ; aucun visiteur n'est bloqué par adresse IP.

UC-12 — Notifier le créateur d'un lien épuisé. Déclencheur : le dernier exemplaire est compté. Un e-mail part vers le créateur du lien (document, nombre d'exemplaires, date et heure). **Exactement un** e-mail par lien, même si l'événement est traité deux fois (BNF-10).

UC-15 — Détecter et bloquer un abus. Déclencheur : un rédacteur atteint le seuil de refus (H-10). Il est bloqué pour une durée fixée, voit l'état de blocage et l'heure de fin, apparaît sur le tableau de bord, et le blocage est levé à l'échéance.

2.7 Parcours

Trois parcours résument les points forts de l'application.

Parcours	Déroulement	Teste
A. Écrire le journal à plusieurs	Camille crée « Journal 2026 » et envoie le lien d'invitation à Idriss et Sarah. Tous trois écrivent, parfois dans le même paragraphe. Idriss colle un long texte avec peu de crédit : le collage est refusé et il voit l'heure de recharge. Sarah supprime un passage par erreur ; Camille, propriétaire, restaure la version précédente.	C-2, C-4
B. La parution	Camille crée un lien de 500 exemplaires. À l'heure annoncée, 15 000 personnes cliquent, dont 12 000 dans la première minute. Chacun reçoit « prêt », « patientez » ou « épuisé ». Une fois les 500 comptés, tous les autres voient « épuisé » et Camille reçoit un e-mail. Pendant ce temps, l'équipe corrige une coquille, que contiennent les exemplaires distribués ensuite, et 300 rédacteurs continuent d'écrire.	C-1, C-3, C-5, C-6
C. L'association surveille	Elle suit le tableau de bord : documents ouverts, lien dont les exemplaires restants baissent, rédacteur bloqué pour avoir insisté sans crédit.	BF-16, BF-17

3 Conception

3.1 Conception architecturale

3.1.1 Principe : deux chemins séparés

Le sujet impose que 15 000 visiteurs cliquent pendant que 300 rédacteurs écrivent. Les deux trafics n'ont pas la même nature, donc ils ne passent pas par les mêmes composants :

— **l'écriture** demande une connexion longue par rédacteur (WebSocket), des documents gardés en mémoire et un ordre cohérent des modifications : un petit serveur dédié, sur une instance EC2 ;
— **le téléchargement** est fait de requêtes courtes, très nombreuses sur un instant très court, sans état entre elles : des fonctions Lambda, qui montent en charge d'elles-mêmes et ne coûtent rien au repos [7].

Le pic des visiteurs ne peut ainsi pas ralentir les rédacteurs : il ne touche jamais le serveur d'édition (C-5). Une seule adresse HTTPS, fournie par CloudFront, sert l'ensemble. Elle évite d'acheter un nom de domaine, donne un certificat gratuit et place l'application, l'API et le temps réel sur la même origine, donc sans configuration de partage entre origines (CORS) [8].

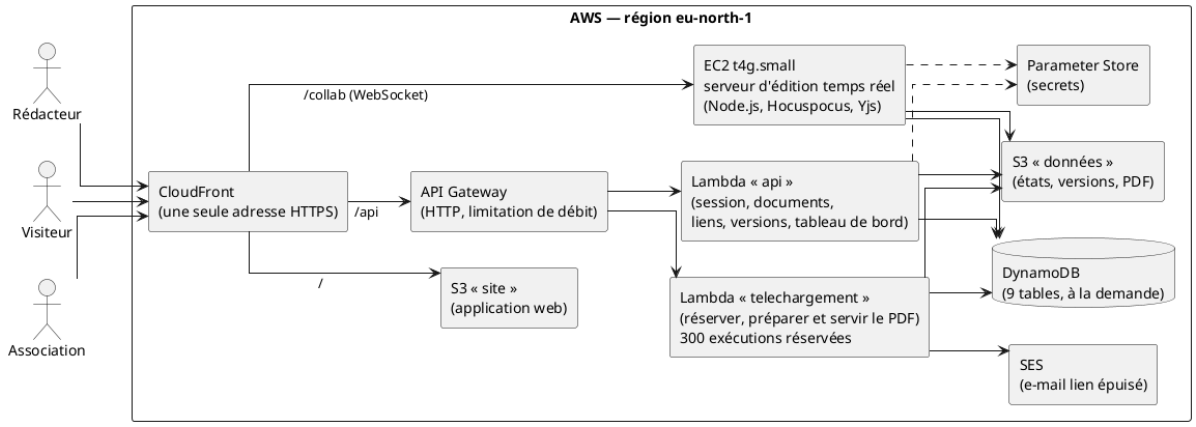


Figure 2 – Architecture de B-SyncText sur AWS

3.1.2 Composants et déploiement

La figure 3.1.2 montre les composants logiciels et leurs interfaces ; la figure 3.1.2 montre où chacun s'exécute. Un module commun (crédit, jetons, numérotation des versions, lecture de l'état Yjs) est partagé par les trois composants serveur, pour qu'une règle ne soit écrite qu'une fois.

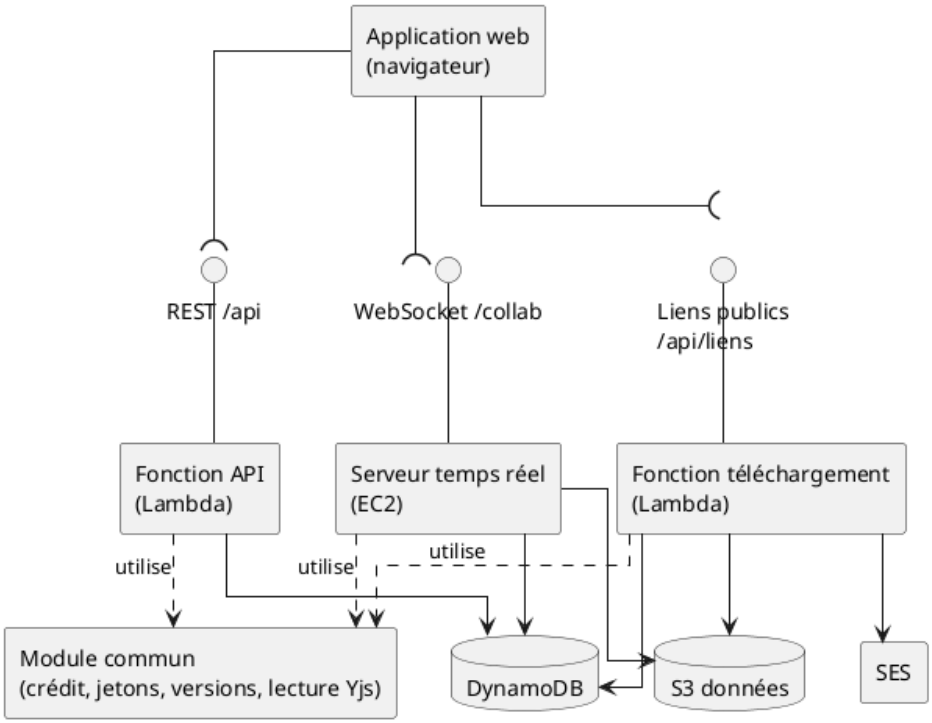


Figure 3 – Composants et interfaces

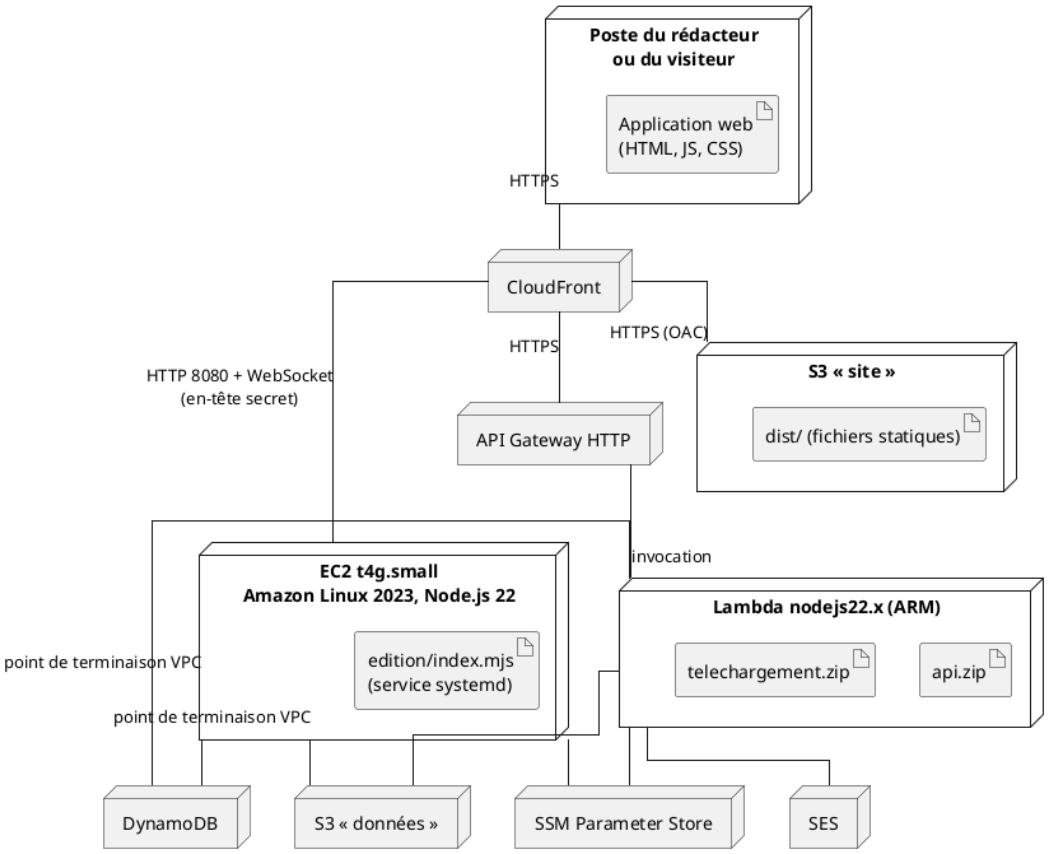


Figure 4 – Déploiement : nœuds, artefacts et liaisons

3.1.3 Services AWS retenus

Service	Rôle	Pourquoi lui
CloudFront	une seule adresse HTTPS ; aiguille `/` vers le site, `/api/*` vers l'API, `/collab` vers le serveur d'édition	certificat sans nom de domaine ; même origine ; gratuit à ce volume
S3 « site »	pages de l'application, bucket privé lu par CloudFront seul [8]	fichiers statiques, coût nul
EC2 `t4g.small`	serveur d'édition temps réel (Node.js, Hocuspocus)	connexions longues, documents en mémoire ; seul composant de l'application facturé à l'heure
EC2 `m7i-flex.large`	générateur de charge, créé pour les tests puis supprimé	reproduit le pic depuis la région de l'application : un poste personnel ne tient pas mille connexions simultanées
API Gateway HTTP + Lambda `api`	session, documents, membres, liens, versions, tableau de bord	requêtes courtes ; rien au repos
Lambda `telechargement`	réserver un exemplaire, préparer et servir le PDF ; 300 exécutions réservées [9]	absorbe le pic automatiquement ; la réserve garantit sa part de capacité
DynamoDB à la demande	toutes les données ; écritures *conditionnelles* pour C-1 et C-4 [10]	opérations atomiques natives ; paiement à la requête
S3 « données »	états compactés des documents, versions, PDF de base (CH-06)	objets de taille libre, peu coûteux
SES	e-mail « lien épuisé »	service d'e-mail d'AWS disponible dans la région [11]
SSM Parameter Store	secrets : signature des sessions, code administrateur, en-tête secret de CloudFront	secrets hors du dépôt, gratuits

3.1.4 Comment chaque contrainte est tenue

Contrainte	Mécanisme	Test
C-1	Le lien porte un compteur `disponibles`. Réserver, c'est le décrémenter par une *écriture conditionnelle* (« si > 0 »), que DynamoDB exécute de façon atomique : 1 000 demandes simultanées ne peuvent pas obtenir plus de N réservations [10]. L'adresse du fichier est à usage unique, consommée par une seconde écriture conditionnelle.	T-01, T-02
C-2	Le document est un CRDT (type de données répliqué sans conflit) : chaque modification a un identifiant unique et les fusions donnent le même texte quel que soit l'ordre d'arrivée [12, 13]. Une modification est *acceptée* quand elle est écrite dans le journal DynamoDB ; ensuite seulement elle est diffusée et confirmée. Après un redémarrage, l'état est reconstruit depuis S3 et le journal.	T-03
C-3	Le PDF est construit à partir du dernier état et du journal, lus au moment de la demande de fichier. Un PDF de base est gardé par état du document.	T-04
C-4	Le crédit du jour est un élément DynamoDB par rédacteur et par jour, décrémenté par écriture conditionnelle (« si *restant* $\geq n$ ») : il est unique quel que soit l'appareil.	T-05
C-5	Les visiteurs ne touchent pas l'EC2. Au-delà de la capacité, la saturation devient « patientez » et la page réessaie : aucun visiteur sans réponse. Le PDF est préparé une seule fois, avant la réservation.	T-06
C-6	Au dernier exemplaire compté, le lien passe à « épuisé », état définitif gardé en mémoire par chaque fonction : toute demande répond sans lire le document. L'e-mail part une fois, grâce à un drapeau posé par écriture conditionnelle.	T-07, T-09

Aucune tâche planifiée. La recharge du crédit est implicite (l'élément du jour est créé à 10 000 au premier usage et expire seul [14]), un blocage porte son heure de fin, et la remise en stock d'une réservation expirée est faite par les visiteurs qui patientent, quand ils redemandent. Il n'y a donc aucun service de plus à payer ni à surveiller.

3.1.5 Dimensionnement du pic

La charge est de 12 000 clics en 60 s, soit 200 par seconde en moyenne. On dimensionne sur une pointe de 1 000 par seconde, cinq fois la moyenne. La simultanéité Lambda vaut le débit multiplié par la durée moyenne d'une requête [7] ; le quota du compte est de 400 exécutions simultanées.

Requête Lambda	Débit (req/s)	Durée estimée (s)	Simultanéité
Réservation	1 000	0,1	100
Envoi du PDF ($N = 500$)	100	0,3	30
Relances des visiteurs qui patientent	1 000	0,05	50
Total	**2 100**		**180**

Ces 180 tiennent dans les 300 exécutions réservées. Les rédacteurs ne consomment pas de quota Lambda : ils passent par l'EC2.

3.1.6 Sécurité et abus

Qui a le droit de quoi. Chaque règle est vérifiée côté serveur ; l'interface ne fait que masquer les boutons.

Action	Autorisé à
lire, écrire, voir l'historique, créer une version, créer un lien, changer le mode de téléchargement	membre du document
donner accès, régénérer l'invitation, nommer un propriétaire, restaurer	propriétaire
retirer le rôle de propriétaire à un autre	propriétaire principal
télécharger	toute personne qui a le lien, sans identité
tableau de bord	détenteur du code administrateur

Droits AWS : un rôle par composant. Chaque composant a son rôle, limité à des ressources nommées, selon le principe du moindre privilège [6]. Aucune clé d'accès n'existe nulle part : le déploiement utilise la session de connexion de la personne qui déploie.

Composant	Peut	Ne peut pas
Serveur d'édition (EC2)	7 tables (crédits, journal, documents…), dossiers `etats/` et `versions/`, lecture des secrets	toucher aux liens, aux PDF, au code administrateur
Lambda `api`	tables métier, lecture des versions, secrets	écrire dans S3, envoyer un e-mail
Lambda `telechargement`	liens, réservations, lecture du journal, dossier `pdf/`, envoi d'e-mail	lire les secrets, toucher aux crédits
CloudFront	lire le bucket du site	tout le reste

Réseau et données.

— Le serveur d'édition n'accepte que le trafic de CloudFront : le groupe de sécurité est limité à la liste d'adresses gérée par AWS [15], et le serveur refuse toute requête sans l'en-tête secret ajouté par CloudFront.
— Aucun port SSH n'est ouvert ; l'administration passe par Session Manager [16]. Les métadonnées d'instance exigent IMDSv2 [17].
— Les buckets S3 sont privés et chiffrés ; DynamoDB est chiffré par défaut. Les identifiants de lien, de ticket et d'invitation sont aléatoires (128 bits).

Abus.

Menace	Parade
rédacteur qui force le crédit ou inonde le serveur	20 refus en une minute : blocage de 15 minutes, visible au tableau de bord
collage géant	document limité à 200 000 caractères ; insertion supérieure au crédit refusée en entier
rafale de clics sur un lien	limitation de débit d'API Gateway [18] ; la réponse 429 devient « patientez »
réutilisation d'une adresse de fichier	ticket à usage unique, consommé par écriture conditionnelle
essais de codes administrateur	10 échecs par minute au plus, puis refus
dépense qui s'emballe	alertes de budget à 5, 10 et 20 USD ; concurrence Lambda plafonnée

3.1.7 Coût

Poste	Remarque	USD
EC2 `t4g.small` et adresse IPv4 publique	environ 0,55 USD par jour si l'on oublie de supprimer	—
Lambda, API Gateway, DynamoDB, S3, CloudFront	en grande partie dans les offres gratuites	—
Estimation avant le projet	jusqu'au 27 octobre	< 5

3.2 Conception détaillée

3.2.1 Classes

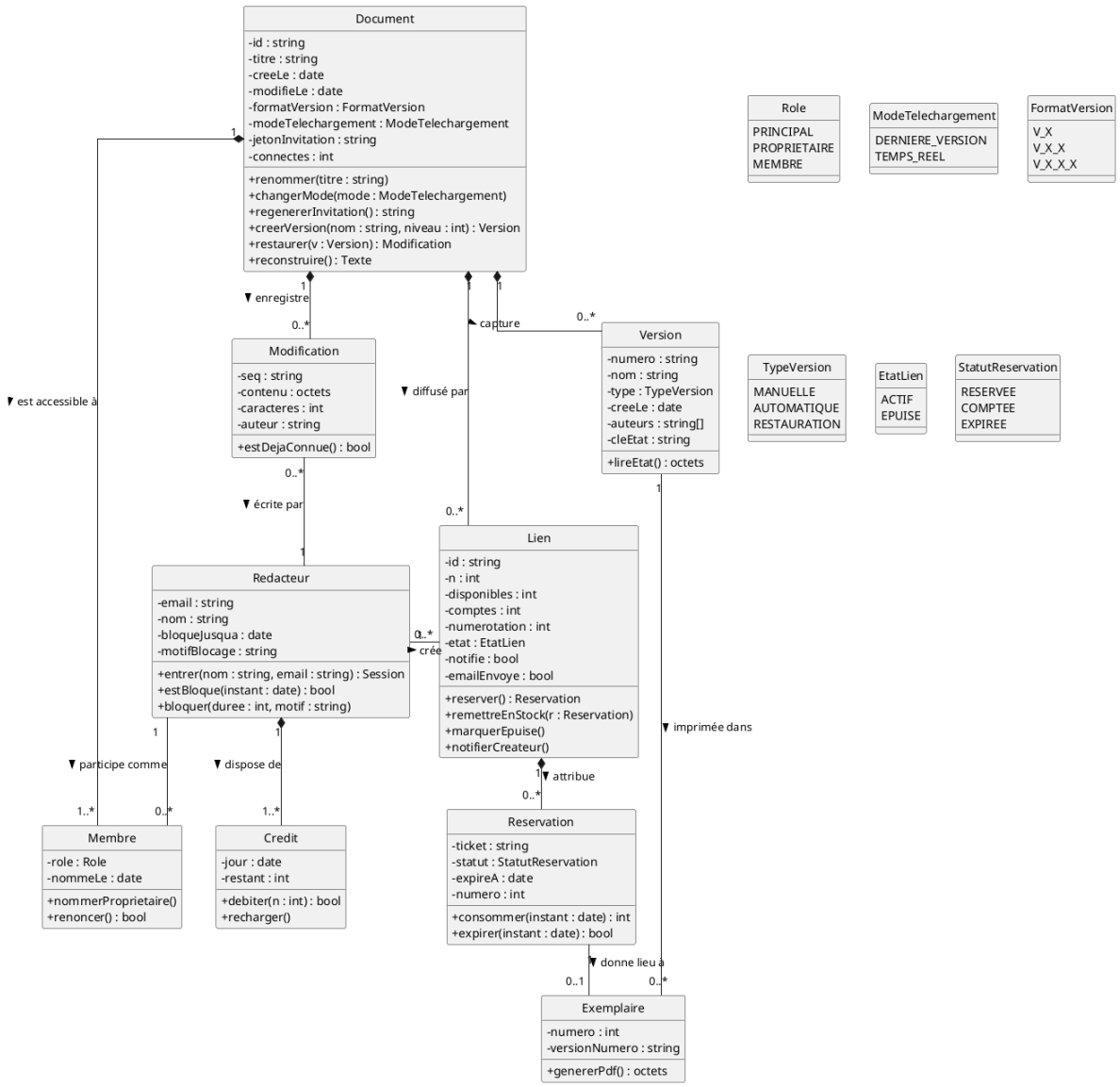


Figure 5 – Diagramme de classes

3.2.2 Données

Table	Contenu	Particularité
Redacteurs	fiche d'un rédacteur : nom, blocage et motif	clé : e-mail
Credits	crédit restant d'un rédacteur pour un jour	expire au bout de 2 jours [14]
Documents	titre, format et compteurs de version, mode de téléchargement, jeton d'invitation	
Membres	rôle d'un rédacteur dans un document	index par e-mail : liste des documents d'un rédacteur
Versions	numéro, nom, type, auteurs, clé de l'état dans S3	
Liens	N, disponibles, comptés, numérotation, état, drapeaux d'e-mail	index par document
Reservations	ticket, statut, expiration, numéro	index sur les réservations en cours
Exemplaire	numéro k et version du PDF livré ; porté par la réservation comptée	inscrit en pied de chaque PDF

Le bucket S3 « données » contient trois dossiers : `etats/` (dernier état compacté de chaque document), `versions/` (les versions) et `pdf/` (un PDF de base par état du document).

Écritures atomiques qui portent les contraintes.

But	Écriture DynamoDB
débiter le crédit, recharge implicite (C-4)	restant $\leftarrow$ restant $- n$, *seulement si* l'élément n'existe pas ou restant $\geq n$
réserver un exemplaire (C-1)	disponibles $\leftarrow$ disponibles $- 1$, *seulement si* disponibles > 0 et lien actif
compter un exemplaire (C-1)	réservation « réservée » $\rightarrow$ « comptée », *seulement si* elle n'a pas expiré ; puis comptés $+ 1$ et numérotation $+ 1$ sur le lien (donne le numéro k)
remettre en stock une réservation expirée	réservation « réservée » $\rightarrow$ « expirée », *seulement si* elle a expiré ; puis disponibles $+ 1$
envoyer un seul e-mail (BNF-10)	drapeau « notifié » mis à vrai, *seulement si* il valait faux
numéroter une version	compteur du niveau choisi incrémenté de façon atomique ; les niveaux à droite repassent à 0

Invariant du lien. À tout instant : *disponibles* $+$ *réservations actives* $+$ *comptés* $= N$.

3.2.3 États

Un *diagramme d'états-transitions* montre les états d'un objet et les événements qui le font passer de l'un à l'autre. Les états terminaux nommés (fond gris) sont les issues possibles d'un parcours.

Lien et réservation (C-1, C-6). Un lien épuisé ne revient jamais à l'état actif. Une réservation est comptée une seule fois, ou bien elle expire et son exemplaire revient dans le stock.

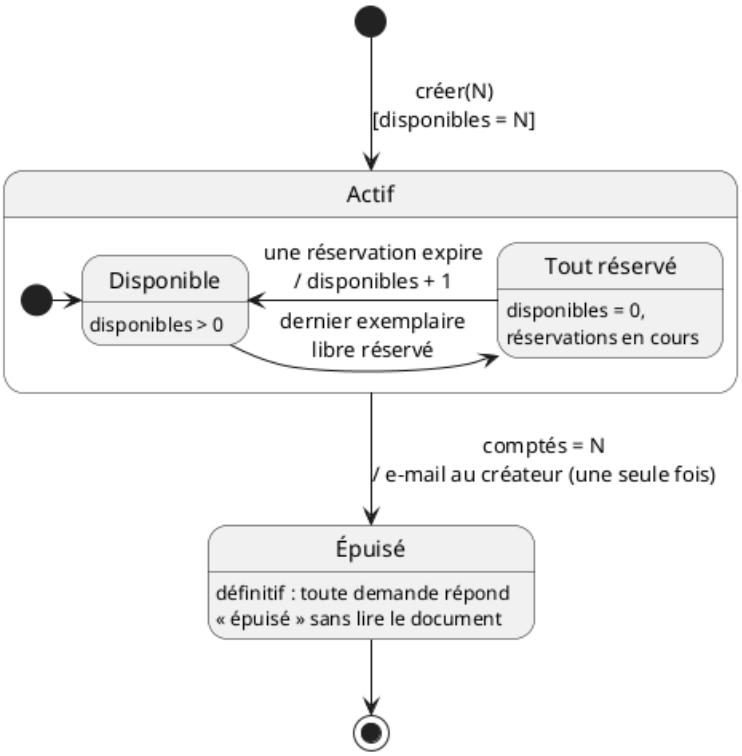


Figure 6 – États d'un lien de téléchargement

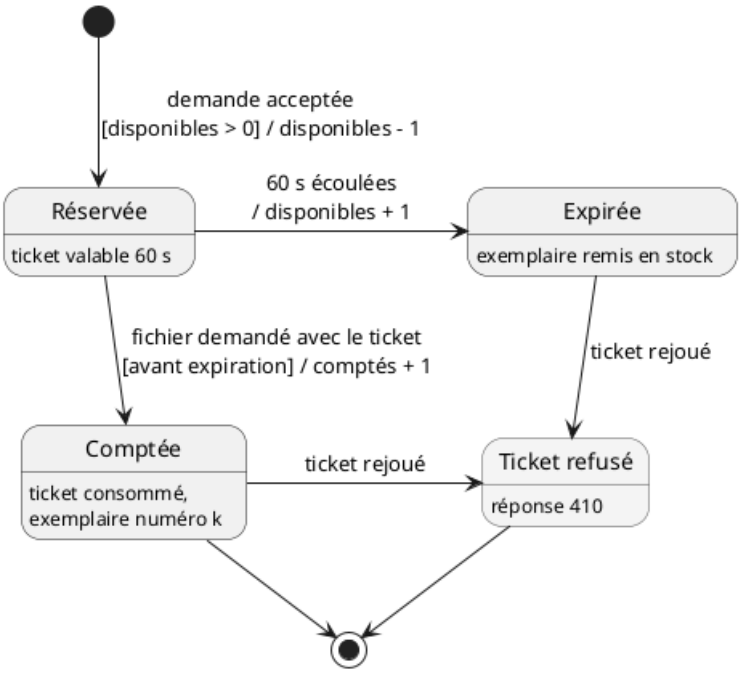


Figure 7 – États d'une réservation

Page de téléchargement (C-5, C-6). Chaque visiteur est toujours dans un état nommé, jamais sans réponse.

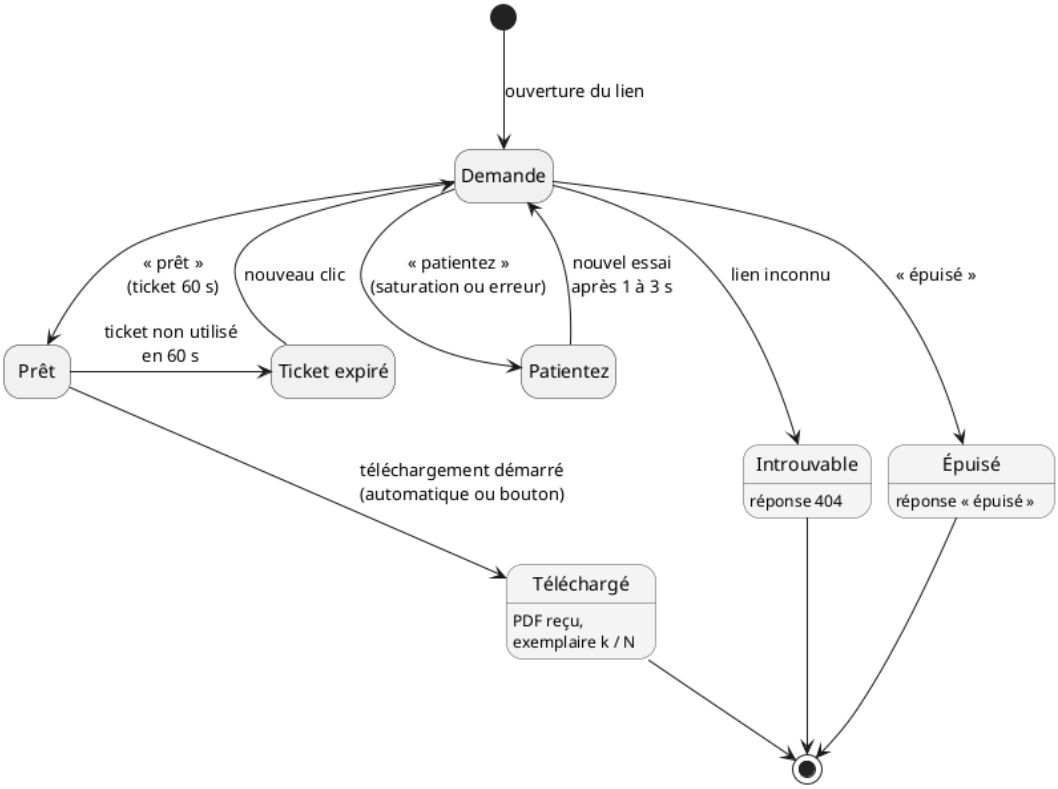


Figure 8 – États de la page publique de téléchargement

Éditeur et rédacteur (C-2, C-4). « Enregistré » ne s'affiche qu'après acceptation par le serveur. Un rédacteur qui a encore du crédit peut être bloqué : vingt refus en une minute suffisent, par exemple en collant plusieurs fois un texte plus long que le crédit restant.

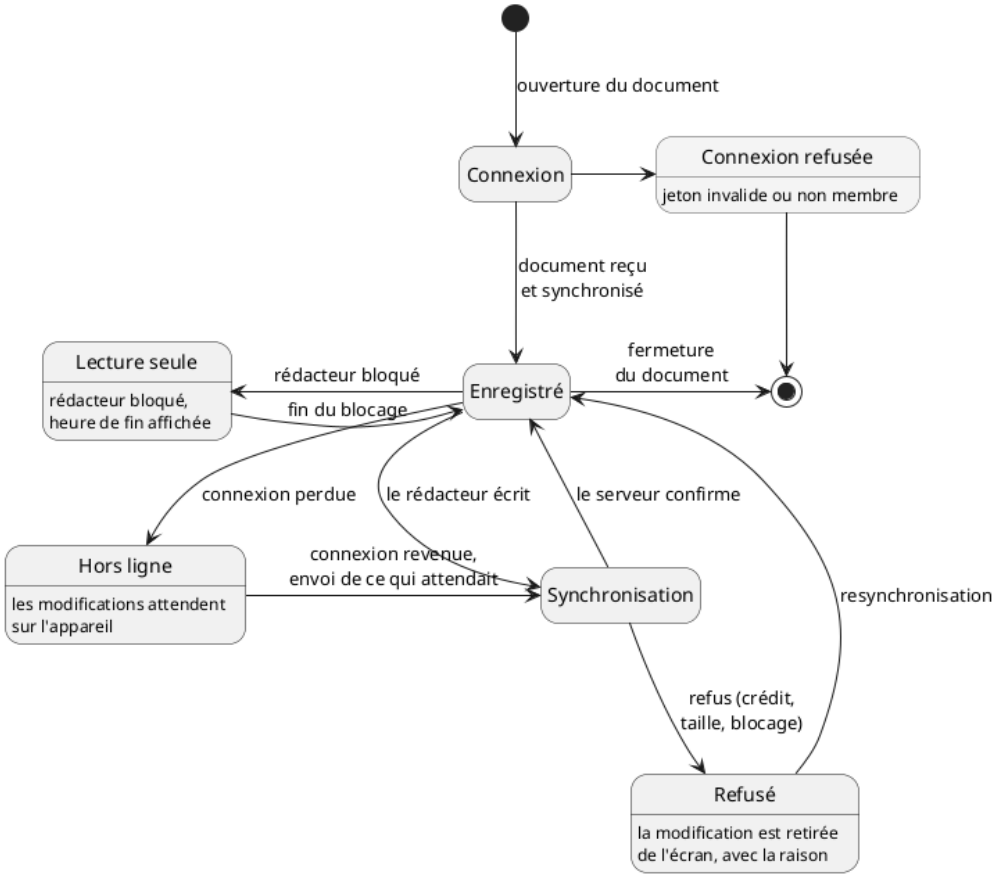


Figure 9 – États de l'indicateur d'enregistrement de l'éditeur

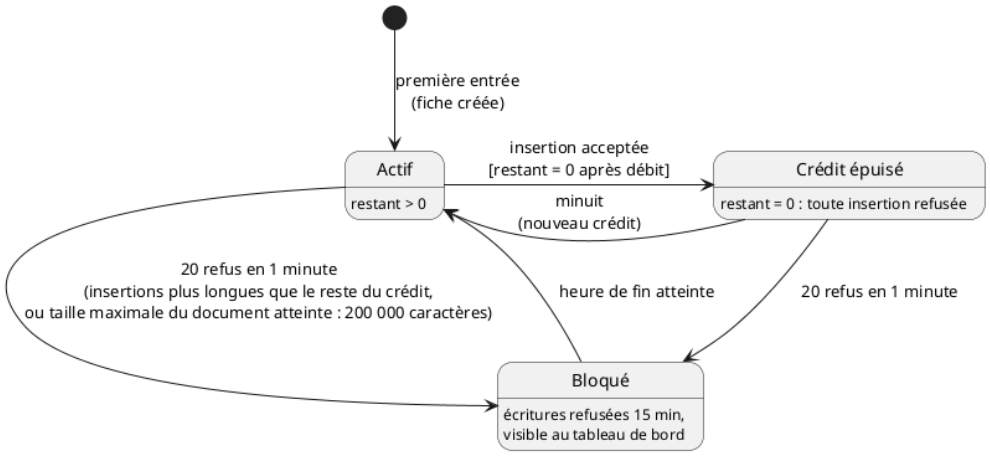


Figure 10 – États d'un rédacteur face aux limites

3.2.4 Séquences

Les diagrammes de séquence montrent l'ordre des échanges. La figure 3.2.4 suit une modification acceptée : elle n'est confirmée qu'après l'écriture dans le journal. La figure 3.2.4 suit un téléchargement, de la réservation au comptage.

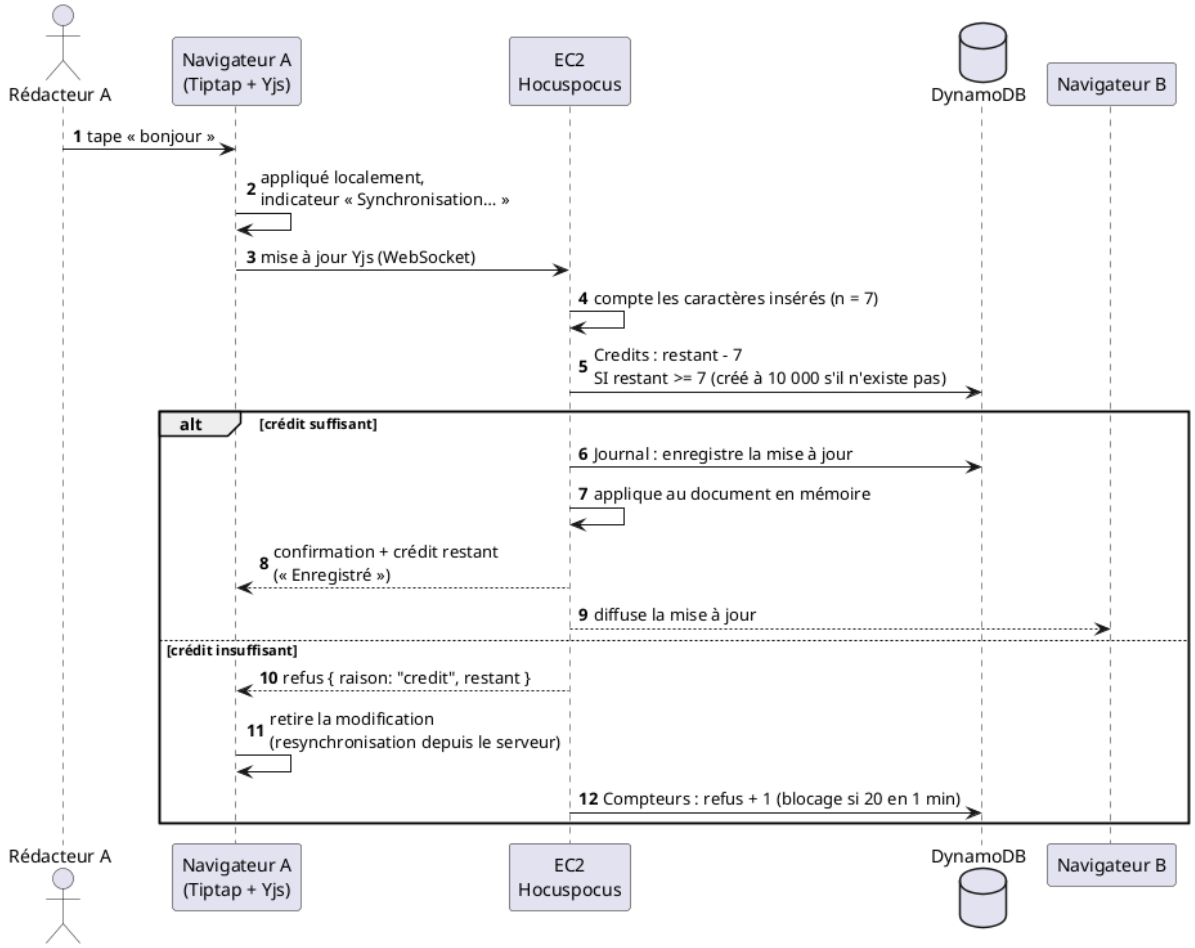


Figure 11 – Séquence : une modification acceptée

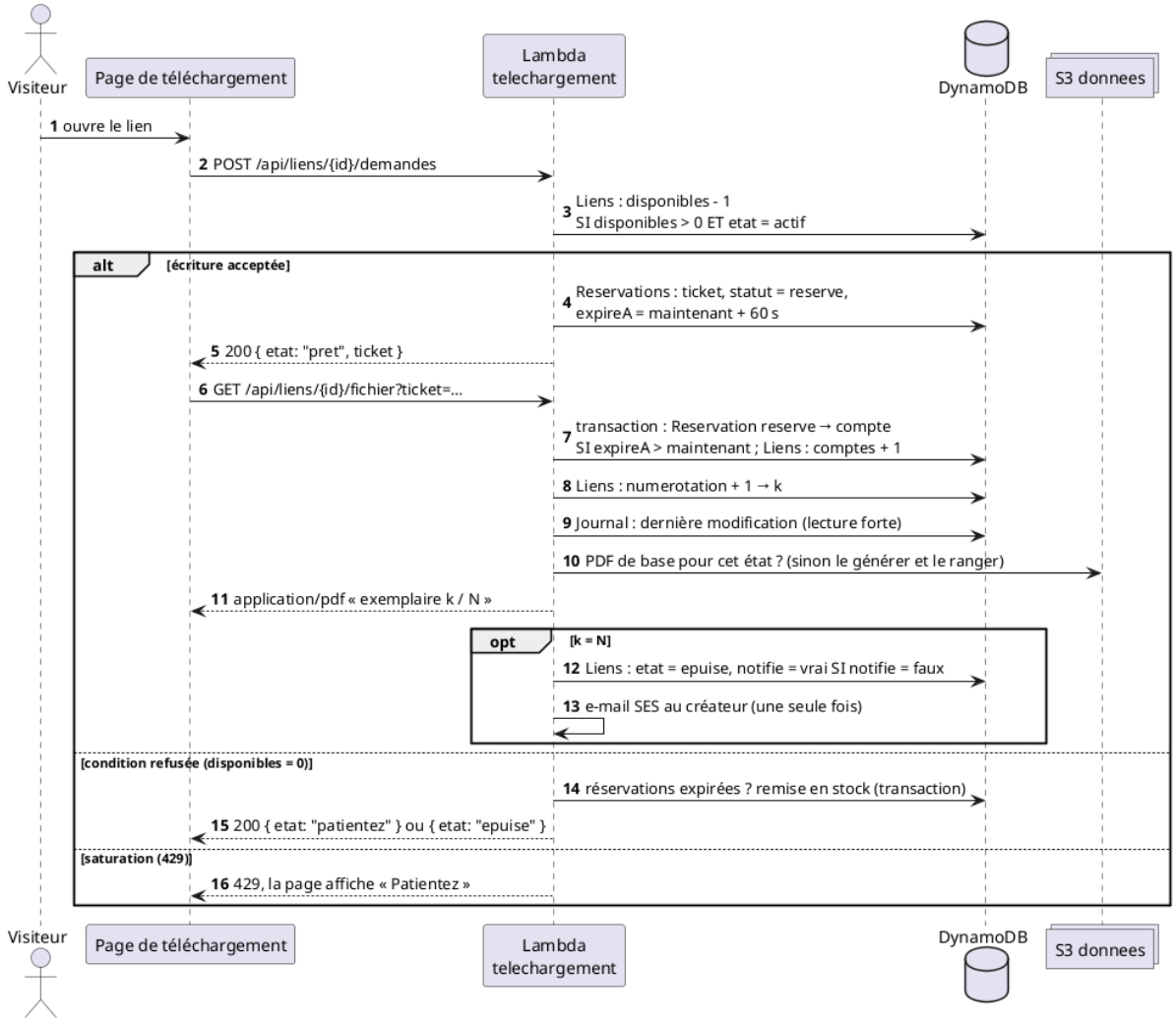


Figure 12 – Séquence : un téléchargement

3.2.5 Identité visuelle

L'application s'appelle **B-SyncText** (B comme l'initiale de mon nom, *Sync* comme synchronisation, *Text* comme texte). Le logo montre une feuille à coin plié, un curseur et six personnes autour, pour les rédacteurs.

Élément	Choix
couleur de marque	orange `#FC7C20`, relevé sur le modèle de PDF
texte sur l'orange	noir d'encre `#201A1B`, contraste 6,57 :1
orange en texte	`#A63F00`, contraste 5,86 :1
« patientez »	bleu
« épuisé »	gris chaud neutre, pas rouge
polices	Manrope (interface), Source Serif 4 (texte des documents), auto-hébergées
page de l'éditeur	format A4 sur une toile grise

4 Choix technologiques

Besoin	Choix	Alternative écartée	Raison
Fusion des écritures concurrentes (C-2)	**CRDT Yjs** [19, 13]	transformation opérationnelle (OT, comme Google Docs) [20] ; sérialisation maison	convergence démontrée dans la littérature [12], bibliothèque éprouvée ; l'OT demande un algorithme de transformation à écrire et à prouver soi-même
Serveur temps réel	**Hocuspocus** (serveur Yjs, Node.js) sur EC2 [21]	API Gateway WebSocket avec Lambda	il faudrait recharger le document à chaque message et diffuser par un appel d'API par connexion : lent et coûteux ; Hocuspocus offre les points d'accroche nécessaires (authentification, contrôle avant d'accepter une modification)
Éditeur de texte	**Tiptap** sur ProseMirror, avec extensions de collaboration et de curseurs [22, 23]	Quill, simple zone de texte	mise en forme de base, curseurs et présence prêts à l'emploi avec Yjs
Application web	**React** et Vite, TypeScript	Vue, HTML sans cadriciel	écosystème de Tiptap
Fonctions et serveur	**Node.js 22** partout, TypeScript	Python	un seul langage ; le même code Yjs sert au serveur d'édition et à la génération du PDF
PDF	**pdfmake** (PDF de base) et **pdf-lib** (numéro d'exemplaire) [24, 25]	navigateur sans interface (Chromium) dans Lambda	bibliothèques légères, démarrage rapide ; Chromium pèse plus de 100 Mo et démarre lentement
Base de données	**DynamoDB à la demande** [10]	RDS PostgreSQL	écritures conditionnelles atomiques (C-1, C-4) ; pas de serveur facturé à l'heure ; RDS coûte même au repos
Diffusion	**CloudFront** [8]	répartiteur de charge ALB	ALB est facturé à l'heure ; CloudFront donne HTTPS sans nom de domaine
Infrastructure en code	**CloudFormation**, avec l'AWS CLI [4]	Terraform, CDK	aucun outil à installer ; l'état est gardé par AWS ; une commande pour tout supprimer
Accès à l'EC2	**Session Manager** [16]	SSH avec fichier de clé	aucun port 22 ouvert, aucune clé à garder
Test de charge	**k6** [5]	JMeter, Artillery	scripts courts en JavaScript, gros volumes depuis une seule machine, résultats chiffrés exportables
Tests de convergence	script Node.js qui pilote N clients Yjs	tests manuels	rejouable et mesurable : chaque client compare son texte à celui des autres

Conclusion

Ce dossier a présenté la conception de B-SyncText, un éditeur de documents partagé déployé sur AWS, pour répondre à la question : comment tenir les six contraintes du sujet avec des services simples et peu coûteux ? La réponse tient en trois idées. L'écriture et le téléchargement passent par deux chemins séparés, donc le pic des visiteurs ne touche pas les rédacteurs. Les contraintes éliminatoires reposent sur des écritures conditionnelles atomiques de DynamoDB, qui garantissent « exactement N » quel que soit le nombre de clics. Les modifications sont des opérations d'un CRDT,

confirmées seulement après leur écriture dans le journal, donc aucune modification acceptée n'est perdue.

Références

[1] *Un éditeur de documents partagé : projet d'évaluation, développement web et cloud*. Sujet du projet. 2026.

[2] *Leçon 2 : Génie logiciel, spécification*. Support de cours, ENSPY.

[3] *Leçon 3 : Génie logiciel, modélisation objets (UML)*. Support de cours, ENSPY.

[4] Amazon Web Services. *AWS CloudFormation User Guide : What is AWS CloudFormation ?* URL : https://docs.aws.amazon.com/AWSCloudFormation/latest/UserGuide/Welcome.html (visité le 26/10/2026).

[5] Grafana Labs. *Grafana k6 documentation : scenarios and executors (arrival-rate load tests)*. URL : https://grafana.com/docs/k6/latest/ (visité le 26/10/2026).

[6] menraromial.com. *Cours AWS : bonnes pratiques (moindre privilège, rôles, IMDSv2, buckets privés, suppression complète)*. Synthèse dans documentation/references/bonnes-pratiques-aws.pdf. 2026. URL : https://menraromial.com/aws-course.

[7] Amazon Web Services. *AWS Lambda Developer Guide : Understanding Lambda function scaling*. URL : https://docs.aws.amazon.com/lambda/latest/dg/lambda-concurrency.html (visité le 26/10/2026).

[8] Amazon Web Services. *Amazon CloudFront Developer Guide : Restricting access to an Amazon S3 origin (origin access control)*. URL : https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/private-content-restricting-access-to-s3.html (visité le 26/10/2026).

[9] Amazon Web Services. *AWS Lambda Developer Guide : Configuring reserved concurrency for a function*. URL : https://docs.aws.amazon.com/lambda/latest/dg/configuration-concurrency.html (visité le 26/10/2026).

[10] Amazon Web Services. *Amazon DynamoDB Developer Guide : Condition and filter expressions, operators, and functions*. URL : https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/Expressions.ConditionExpressions.html (visité le 26/10/2026).

[11] Amazon Web Services. *Amazon SES Developer Guide : Request production access (moving out of the Amazon SES sandbox)*. URL : https://docs.aws.amazon.com/ses/latest/dg/request-production-access.html (visité le 26/10/2026).

[12] Marc Shapiro et al. « Conflict-free Replicated Data Types ». In : *Stabilization, Safety, and Security of Distributed Systems (SSS 2011)*. T. 6976. Lecture Notes in Computer Science. Springer, 2011, p. 386-400.

[13] Petru Nicolaescu et al. « Near Real-Time Peer-to-Peer Shared Editing on Extensible Data Types ». In : *Proceedings of the 19th International Conference on Supporting Group Work (GROUP '16)*. ACM, 2016, p. 39-49.

[14] Amazon Web Services. *Amazon DynamoDB Developer Guide : Time to Live (TTL)*. URL : https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/TTL.html (visité le 26/10/2026).

[15] Amazon Web Services. *Amazon CloudFront Developer Guide : Limit access to your origins using the AWS-managed prefix list*. URL : https://docs.aws.amazon.com/AmazonCloudFront/latest/DeveloperGuide/LocationsOfEdgeServers.html (visité le 26/10/2026).

[16] Amazon Web Services. *AWS Systems Manager User Guide : Session Manager*. URL : https://docs.aws.amazon.com/systems-manager/latest/userguide/session-manager.html (visité le 26/10/2026).

[17] Amazon Web Services. *Amazon EC2 User Guide : Use IMDSv2*. URL : https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/configuring-instance-metadata-service.html (visité le 26/10/2026).

[18] Amazon Web Services. *Amazon API Gateway Developer Guide : Throttle requests to your HTTP APIs*. URL : https://docs.aws.amazon.com/apigateway/latest/developerguide/http-api-throttling.html (visité le 26/10/2026).

[19] Yjs contributors. *Yjs : a CRDT framework for shared editing*. URL : https://docs.yjs.dev/ (visité le 26/10/2026).

[20] Clarence A. Ellis et Simon J. Gibbs. « Concurrency Control in Groupware Systems ». In : *Proceedings of the 1989 ACM SIGMOD International Conference on Management of Data*. ACM, 1989, p. 399-407.

[21] Tiptap GmbH. *Hocuspocus : a plug and play collaboration backend based on Y.js*. URL : https://tiptap.dev/docs/hocuspocus (visité le 26/10/2026).

[22] Tiptap GmbH. *Tiptap : the headless editor framework (collaboration and collaboration cursor extensions)*. URL : https://tiptap.dev/docs/editor (visité le 26/10/2026).

[23] Marijn Haverbeke. *ProseMirror : a toolkit for building rich-text editors*. URL : https://prosemirror.net/docs/ (visité le 26/10/2026).

[24] pdfmake contributors. *pdfmake : client/server side PDF printing in pure JavaScript*. URL : https://pdfmake.github.io/docs/ (visité le 26/10/2026).

[25] pdf-lib contributors. *pdf-lib : create and modify PDF documents in any JavaScript environment*. URL : https://pdf-lib.js.org/ (visité le 26/10/2026).